

Days 1-5 — Python for data science: complete study guide

MD Mutasim Billah · 52-week Data Science to ML/AI roadmap · Week 1 foundations — definitions, real-world examples, implementations, and an interview-style quiz

This guide follows the same format as the Week 3 review: for each day, a plain-language definition, a real-world analogy, and a runnable implementation snippet, followed by a technical and theoretical quiz written the way these fundamentals tend to come up in a data science interview. If any day below doesn't match what you actually covered, tell me which day is off and I'll correct it against your real pfdsoX.py files.

Day 1 — Variables and data types

Definition

Definition — Variable: a named reference to a value stored in memory. Python is dynamically typed, meaning a variable's type is determined by the value assigned to it, and can change if reassigned.

Definition — Core data types: int (whole numbers), float (decimals), str (text), bool (True/False) — the building blocks every dataset eventually gets loaded into, whether that's a single value or a column of a DataFrame.

Real-world example

Real-world example: A spreadsheet column of ages is all int, a column of prices is float, a column of names is str. Before pandas ever enters the picture, understanding that a CSV's "Age" column might load as float64 (because of missing values) instead of int64 explains a whole class of confusing bugs later.

Implementation

```
age = 29                # int
fare = 71.28            # float
name = "Mutasim"        # str
is_survivor = True      # bool

print(type(age), type(fare), type(name), type(is_survivor))

# type conversion — common in cleaning raw data
age_str = str(age)
fare_int = int(fare)     # truncates to 71, does not round
```

- `type()` — returns a value's actual type at runtime; useful for debugging when a column behaves unexpectedly (e.g. numbers stored as strings after a CSV read)

- `int(fare)` truncates rather than rounds — 71.28 becomes 71, not rounded to the nearest whole number; `round(fare)` would give the mathematically rounded value instead

Day 2 — Control flow: conditionals and loops

Definition

Definition — Conditional (if/elif/else): branches code execution based on whether a condition evaluates to True or False.

Definition — Loop (for / while): repeats a block of code either a fixed number of times over an iterable (for) or until a condition becomes False (while).

Real-world example

Real-world example: Bucketing passengers into age groups ("child", "adult", "senior") before pandas even enters the picture is just a chain of if/elif/else statements. Looping over a list of filenames to process each one is the same for-loop pattern used later to loop over model candidates in Day 19.

Implementation

```
def age_group(age):
    if age < 13:
        return "child"
    elif age < 60:
        return "adult"
    else:
        return "senior"

ages = [8, 34, 67, 22]
for a in ages:
    print(a, "->", age_group(a))

count = 0
while count < 3:
    print("iteration", count)
    count += 1
```

- elif conditions are checked top to bottom, and only the first True branch runs — order matters when ranges could overlap
- while loops require the loop variable to be updated inside the loop body (`count += 1`) or the loop never terminates — a classic beginner infinite-loop bug

Day 3 — Functions

Definition

Definition — Function: a reusable, named block of code that accepts inputs (parameters), performs an action, and optionally returns a value. Functions are how every later script in this roadmap stays organized instead of being one long script.

Definition — `*args` and `**kwargs`: special parameters that let a function accept a variable number of positional arguments (`*args`, collected into a tuple) or keyword arguments (`**kwargs`, collected into a dictionary), without the caller having to match an exact fixed signature.

Real-world example

Real-world example: Every day's practice script in this roadmap is built from functions like `load_and_clean()`, `run_cross_validation()`, `save_model()` — each one a self-contained step that can be tested, reused, and read independently, instead of one unbroken block of code from top to bottom.

Implementation

```
def summarize(name, score, passed=True):
    status = "passed" if passed else "failed"
    return f"{name} scored {score} and {status}"

print(summarize("Mutasim", 91))
print(summarize("Guest", 40, passed=False))

def total(*args, **kwargs):
    print("positional:", args)
    print("keyword:", kwargs)

total(1, 2, 3, label="week1", verified=True)
```

- `passed=True` in the function signature is a default argument — callers can omit it entirely and get that default, or override it by name
- `*args` collects any extra positional arguments into a tuple; `**kwargs` collects any extra keyword arguments into a dictionary — this is exactly the pattern scikit-learn model constructors use internally to accept many optional hyperparameters

Day 4 — Data structures and comprehensions

Definition

Definition — list / dict / tuple / set: list: ordered, mutable collection. dict: key-value mapping, unordered before Python 3.7, insertion-ordered since. tuple: ordered, immutable collection. set: unordered collection of unique values, useful for de-duplication and membership testing.

Definition — Comprehension: a compact syntax for building a list, dict, or set from an iterable in a single expression, often replacing a multi-line for-loop that appends to an empty collection.

Real-world example

Real-world example: A dictionary mapping passenger class numbers to readable labels ({1: "First", 2: "Second", 3: "Third"}) is exactly the `class_lookup` table used in Day 11's SQL join practice. A set is the fastest way to answer "which unique embarkation ports appear in this dataset?" without manually removing duplicates.

Implementation

```
ages = [8, 34, 67, 22, 5]

# comprehension instead of a manual loop + append
adults = [a for a in ages if a >= 18]
squared = [a ** 2 for a in ages]

class_lookup = {1: "First", 2: "Second", 3: "Third"}
labels = {k: v.upper() for k, v in class_lookup.items()}

ports = ["S", "C", "Q", "S", "S", "C"]
unique_ports = set(ports)  # {'S', 'C', 'Q'}
```

- `[a for a in ages if a >= 18]` — the `if` at the end filters which items are included; a comprehension without it just transforms every item
- `{k: v.upper() for k, v in class_lookup.items()}` — dict comprehensions need both a key and a value expression separated by `:`, built the same way `pd.get_dummies`-style column mappings get constructed by hand when needed

Day 5 — File handling and exception handling

Definition

Definition — File I/O: reading from or writing to files on disk, typically using a context manager (with `open(...)` as `f`) so the file is automatically closed even if an error occurs partway through.

Definition — Exception handling (`try/except`): catches errors at runtime so a script can respond gracefully (log a message, use a fallback value, retry) instead of crashing outright.

Real-world example

Real-world example: Every script from Day 10 onward starts with `'if not os.path.exists(path): download it'` — that's file-existence checking, the same category of skill as file I/O. A `try/except` around a risky operation (like reading a possibly-missing or malformed file) is what keeps a data pipeline from dying on one bad input instead of processing the rest.

Implementation

```
with open("notes.txt", "w") as f:
    f.write("Day 5 complete\n")

with open("notes.txt", "r") as f:
    content = f.read()
    print(content)

def safe_divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        print("Cannot divide by zero – returning None")
        return None

print(safe_divide(10, 2))
print(safe_divide(10, 0))
```

- `with open(...) as f` — a context manager; the file is guaranteed to close when the block ends, even if an exception is raised inside it, unlike manually calling `f.close()` at the end
- `except ZeroDivisionError` — catching a specific exception type is better practice than a bare `except:`, which would silently swallow every kind of error, including genuine bugs you'd want to know about

Interview prep quiz — technical

Questions about syntax, behavior, and implementation details for Python fundamentals.

Q: What's the difference between a list and a tuple, and when would you choose one over the other?

A: Both are ordered collections, but a list is mutable (can be changed after creation) and a tuple is immutable. Tuples are used when the data shouldn't change (like a fixed coordinate pair) and can also be used as dictionary keys, which lists cannot.

Q: What does a dictionary comprehension look like, and how is it different from a list comprehension?

A: `{key_expr: value_expr for item in iterable}` — it needs two expressions separated by a colon (a key and a value) instead of the single expression a list comprehension uses.

Q: What happens if you don't update the loop variable inside a while loop?

A: The condition never becomes False, so the loop runs forever — an infinite loop. This is one of the most common beginner bugs and a classic thing to check when debugging a script that hangs.

Q: What's the difference between `*args` and `**kwargs` in a function definition?

A: `*args` collects any extra positional arguments into a tuple; `**kwargs` collects any extra keyword arguments into a dictionary. Both let a function accept a flexible number of inputs without a fixed signature.

Q: Why use a context manager (with `open(...)` as `f`) instead of calling `open()` and `close()` manually?

A: The context manager guarantees the file is closed automatically when the block exits, even if an exception is raised partway through — calling `close()` manually means it gets skipped if an error happens before that line runs.

Q: What's the difference between `except ZeroDivisionError` and a bare `except:`?

A: `except ZeroDivisionError` only catches that specific error type, letting other bugs surface normally. A bare `except:` catches every exception, including ones you didn't anticipate, which can hide real bugs instead of fixing them.

Q: How would you remove duplicate values from a list of ports like `['S','C','Q','S','S','C']`?

A: Convert it to a set: `set(ports)` — sets only store unique values by definition, which is faster and simpler than manually looping and checking for duplicates.

Q: Why might a numeric column load as `float64` instead of `int64` when reading a CSV with pandas?

A: If the column contains any missing values (NaN), pandas can't represent NaN in an integer column, so it upgrades the whole column to `float64`, since NaN is a valid float.

Interview prep quiz — theoretical / conceptual

Questions about why these fundamentals matter and how they show up in real data work.

Q: Why does organizing a script into functions matter for a data science project, beyond just style preference?

A: Functions make each step testable and reusable in isolation — you can rerun just the cleaning step without re-running the whole pipeline, and each function's inputs/outputs are explicit, which makes bugs far easier to isolate than in one long unbroken script.

Q: Why is Python's dynamic typing both convenient and a potential source of bugs in data pipelines?

A: Convenient because you don't have to declare types upfront, which speeds up exploratory work. Risky because a column or variable can silently hold an unexpected type (e.g. numbers stored as strings after a bad CSV parse), and Python won't catch that until code actually tries to use it the wrong way.

Q: When would you reach for a comprehension instead of a regular for-loop, and when might a for-loop be the better choice?

A: Comprehensions are ideal for simple, single-expression transformations or filters — they're concise and often faster. A regular for-loop is better when the logic involves multiple steps, side effects (like printing or logging), or conditions that would make a comprehension hard to read.

Q: Why does defensive error handling (try/except) matter more in a data pipeline than in a one-off script?

A: A pipeline that processes many files or rows will eventually hit a bad or missing input. Without error handling, one bad row crashes the entire run; with it, the pipeline can log the issue and continue, which matters enormously when reprocessing everything is expensive or the pipeline runs unattended.